

Guarantee C — Substrate Changes Are Addressable: Standalone Treatment of Change Addressability in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 5, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the five guarantees that compose the CKS determinism contract — **Guarantee C, the change-addressability guarantee** — as a standalone architectural commitment with independent operational content, separable from the other four guarantees with which it composes.

Abstract

The CKS determinism contract names five guarantees as jointly necessary for the deterministic coordination behavior the source paper commits to. A separate integrating-frame note enumerates the five guarantees; subsequent decomposition notes formalize each as standalone. This note formalizes Guarantee C: the commitment that every change to substrate is itself substrate content with provenance, addressable through the substrate's standard read operations. Path retraceability, substrate-only paths, accountability question answers, and regression testing all depend on this content. If changes are tracked externally — in audit logs, event streams, change-data-capture pipelines — the downstream commitments fragment across systems and cannot be operated from substrate alone. The note states the guarantee's four operational components, distinguishes it from four adjacent change-tracking patterns commonly conflated with it, names the failure modes that violate it, and provides an operational test for whether a system's change-addressability architecture satisfies the guarantee.

1. Why Guarantee C needs to be formalized as standalone

The CKS determinism contract is a foundational commitment of the source paper (§4.1, §6.2, §11.3): substrate state behavior is deterministic at the coordination layer in a precisely circumscribed way. A separate foundational note formalizes the contract; an integrating-frame note enumerates its five guarantees and the allowed non-determinism that bounds each. Standalone notes have already formalized Guarantee A (read determinism) and Guarantee B (cell-behavior determinism); this note formalizes Guarantee C (change addressability), with subsequent notes treating Guarantees D (conflict-state preservation) and E (substrate as source of truth).

The integrating frame is correct as far as it goes, and this note does not contradict it. But it leaves Guarantee C architecturally underspecified for cases where the change-addressability content matters in its own right. Three motivating cases anchor the standalone treatment.

The first is path retraceability. The source paper commits substrate to carry path information sufficient to reconstruct, for any current substrate state, the sequence of antecedent states and the rule applications that produced it (§3.1). Reconstruction proceeds by following changes; each step in a path is a change connecting one substrate state to the next. If changes are not addressable as substrate content, reconstruction must traverse external infrastructure, and the substrate-only-paths property is no longer realizable.

The second is regression testing. A regression test that verifies a substrate transition references the change that effected the transition. If changes are addressable in substrate, the test queries them through the same read operations that access any other substrate content; if changes are external, verification fragments across systems.

The third is governance review. A human exercising authority under the CKS human-governed commitment may need to examine what changed in substrate over a time interval — what was decided, when, by whom, under what authority, with what rationale. The change-related answers come from change records that must be substrate-resident and addressable for the inspect right to reach them through standard read operations.

The remedy in each case is the same: name change addressability as a standalone architectural commitment, distinct from the joint determinism contract and the other four guarantees.

2. The Guarantee C commitment, defined precisely

In the CKS pattern, a substrate satisfies **Guarantee C** when every change to the substrate is committed as addressable substrate content with provenance, queryable through the substrate's standard read operations. The commitment has four operational components.

(a) Changes-as-content. Every change to substrate is itself substrate content — not a side-effect, not an external log entry, not an implicit transformation. The change is the architectural object that records what changed; subsequent queries can retrieve it as substrate content alongside the post-change state it produced. The change content carries the new content (what was written) together with the provenance — writer, timestamp, antecedent reference, rule reference, rationale where applicable, relationship metadata where applicable — that the source paper's six-field provenance commitment specifies. The detailed treatment of those fields belongs to the dedicated provenance note.

(b) Atomic-commit-with-provenance. Changes are committed atomically with their provenance per the boundary-crossings architecture that governs writes from cell into substrate. The change content and the provenance are committed together as a single substrate operation; partial commits — content without provenance, or provenance without content — are forbidden. Atomicity is what makes the change addressable as a coherent substrate event rather than as a pair of loosely associated records that may or may not match.

(c) Addressable-identifiers. Each change has an addressable identifier resident in substrate. The identifier permits the change to be referenced from other substrate content (as antecedent references in subsequent writes), queried directly through substrate read operations, and traced through paths. The identifier is substrate content, not an external system-assigned identifier; it persists with the change and is durable to the same standard as any other substrate content.

(d) Durable-persistence. Changes persist in substrate per the substrate-layer persistence commitment. The change record is not discarded after the change takes effect, not summarized into aggregate statistics that lose the original content, not archived to external systems while substrate retains only the post-change state. The change persists with the same durability as the substrate content it modified, for whatever retention window the deployment authorizes.

The four components together define Guarantee C architecturally. Failing any one — even with the others robustly satisfied — fails the guarantee.

3. What the guarantee does NOT claim

Stating precisely what Guarantee C does not claim keeps the standalone framing from drifting into something stronger than the source paper supports.

It does not claim that changes are physically free or arbitrarily fast. Changes have operational cost; the architectural commitment is to change-content addressability, not to change-performance properties. Whether the substrate's change ingestion sustains a given throughput is a deployment-engineering concern outside the architectural commitment.

It does not require append-only architectures. Deployments may use append-only patterns (every change appended as a new substrate event) or update-in-place patterns (changes apply to existing content while change records persist alongside). Either approach satisfies Guarantee C if the four components hold.

It does not require all change history to be retained indefinitely. Deployments may have retention policies that eventually remove old change records. The guarantee applies to changes during their substrate residence; what happens after retention expiration is a deployment policy.

It does not specify implementation patterns. Implementations may use event-sourced architectures, immutable-data structures, versioned storage, append-only logs in substrate, or other patterns. The architectural commitment is to the components being operationally satisfied; specific implementations are deployment choices.

It does not foreclose external change-tracking infrastructure. Deployments may have audit dashboards, observability platforms, or change-feed systems that consume substrate change content for adjacent purposes. The architectural commitment is that these external systems are derivative views; the substrate's own change record is the authoritative source.

It does not require global ordering of changes. Changes may be partially ordered (per substrate scope, per entity, per cell context) rather than globally ordered. Path retraceability follows antecedent references rather than a global timeline; partial order is sufficient for the architectural commitment.

4. What Guarantee C is NOT

Four adjacent change-tracking patterns are commonly conflated with Guarantee C. Each is a real and reasonable commitment in some other architecture; naming what Guarantee C is not is what prevents the misreading.

Not event sourcing. Event sourcing is an architectural pattern in which state is reconstructed from a sequence of events; events are first-class architectural objects in event-sourced architectures. Guarantee C may operationalize through event-sourced patterns, but it does not require event sourcing. The architectural difference is positional: event-sourcing-as-authoritative makes substrate a derivation of an event stream, whereas Guarantee C positions substrate as authoritative (per Guarantee E), with changes as substrate content alongside current state. A CKS substrate may be implemented event-source-style internally, but the substrate — not an external event stream — remains the source of truth.

Not audit logging. Audit logging records events external to the substrate, typically tracking access events and operational metadata for after-the-fact review. Audit logs may include records about substrate changes, but they are external to substrate and do not satisfy Guarantee C when the change record itself is not substrate content. The architectural difference is that audit logs are derivative observations about substrate operations; Guarantee C requires changes to be in substrate as the authoritative record.

Not version control. Version control systems record file changes as commits with metadata. Version control may operationalize substrate change tracking, but version-controlled substrate satisfies Guarantee C only if changes are addressable as substrate content per the four components, not only through the version-control system's interface. The architectural difference is layer: version control is typically an external infrastructure layer; Guarantee C requires changes to be at the substrate-content layer where standard read operations access them.

Not change-data-capture (CDC). CDC pipelines extract changes from databases and propagate them to downstream systems. CDC may produce derivative views of substrate changes for adjacent components — useful for indexing, observability, or cross-system synchronization — but the captured stream is derivative. The substrate's own change record must satisfy Guarantee C independently; the CDC stream is an export, not a substitute.

5. Why Guarantee C is load-bearing for downstream commitments

Naming Guarantee C as standalone makes its load-bearing role visible across several CKS commitments. The role at each is architectural: the downstream commitment depends on the change-addressability content for its own operational content.

The integrating determinism contract depends on Guarantee C as one of its five jointly-necessary guarantees; the contract cannot operate at the change-event level without it. The path retraceability commitment depends on Guarantee C because path reconstruction follows changes from current state back to antecedents; if changes were not addressable in substrate, reconstruction would have to traverse external infrastructure. The substrate-only-paths property depends on Guarantee C because paths are substrate-only only if the changes that compose them are substrate-resident. The accountability-question answers — what was decided, by whom, under what authority, with what rationale — are answered from substrate content; the change-related answers come from addressable change records. The regression-testing capability depends on Guarantee C because regression tests reference change records as substrate content for transition verification. The conflict-as-first-class commitment depends on Guarantee C because conflict events are addressable substrate content and conflict resolutions are themselves addressable changes connecting a conflict-state to the post-resolution state.

These are not new commitments. Each is named in the source paper or in a parent foundational note. Naming them here makes visible that Guarantee C is what these commitments rest on at the change-event level.

6. Failure modes that violate the guarantee

The following failure modes name ways an implementation can fail Guarantee C as a standalone architectural commitment. Each names which of the four components fails.

(a) External-log-only changes. Substrate carries only post-change state; the change record resides in an external audit log and is not substrate-addressable. Component (a) fails.

(b) State-without-change-record. Substrate updates state in place without recording the change as substrate content. The post-change state is present but the change event is not addressable. Component (a) fails.

(c) Provenance-stripping. Changes are committed as substrate content but without the full provenance. Addressability is incomplete because the metadata that makes the change traceable is absent. Component (a) fails — provenance is part of the change content the component requires.

(d) Non-atomic content-and-provenance commits. Content and provenance are committed separately, with windows where one is committed and the other is not. Readers may observe partial commits. Component (b) fails.

(e) External-identifier-only addressability. Changes are identifiable through external systems (audit-log IDs, workflow handles, monitoring identifiers) but not through substrate's own identifier scheme. Addressability depends on external systems and is brittle to their availability. Component (c) fails.

(f) Change-record-summarization. The implementation periodically summarizes change records into aggregate statistics, losing the original content. After summarization, the original change is no longer addressable as substrate content. Component (d) fails.

(g) Change-archival-external. Recent changes persist in substrate; older changes are archived to external systems. The architectural commitment holds only for the recent window; retraceability fragments at the archival boundary. Component (d) fails for the archived window.

(h) Implicit-transformation changes. Operations apply transformations to substrate content as side effects (automatic format conversions, derivation updates, cascade modifications) without recording them as addressable changes. Component (a) fails for the implicit transformations.

(i) Change-flow-bypass. The implementation provides alternative change paths that bypass the boundary-crossings architecture (direct database writes, infrastructure-level state modifications, administrative override interfaces). Components (a) through (d) all fail for bypass-path changes.

(j) Change-merging-without-preservation. The implementation merges multiple changes into a single substrate write for performance optimization, losing the addressability of the individual changes. The merged write is addressable but the individuals — and antecedent references that depended on them — are not. Component (c) fails for the unmerged individuals.

These ten are not exhaustive; they are the modes that recur in deployments under pressure to integrate with external change-tracking infrastructure.

7. Operational test

A system satisfies Guarantee C if and only if all of the following are true at all times during the substrate's existence.

1. Every change to substrate is committed as substrate content with the change content and provenance per the source paper's six-field provenance commitment, satisfying component (a) of §2.
2. Changes are committed atomically — content and provenance together, never separately — satisfying component (b).
3. Each change has a substrate-resident addressable identifier permitting the change to be referenced from other substrate content, queried through substrate read operations, and traced through paths, satisfying component (c).
4. Changes persist with the same durability as any other substrate content for the deployment's retention window, satisfying component (d).
5. Changes are queryable through the standard substrate read operations that access any other substrate content; no separate access mechanism is required for change records.
6. No alternative change paths exist that bypass the boundary-crossings architecture; all changes — including those originating from administrative interfaces, infrastructure-level operations, or automated transformations — produce addressable substrate change records through the same architecture.

A system that fails any of (1)–(6) does not satisfy Guarantee C in the architectural sense, even if its change-tracking functions operationally through external mechanisms.

Conclusion

Implementations under pressure to integrate with enterprise change-tracking infrastructure consistently drift toward external change-tracking patterns. The drift is steady because the external infrastructure — audit logs, event streams, observability platforms — is operationally familiar, well-tooled, and organizationally established as the locus of change visibility. Substrate-resident change addressability appears redundant when external systems are already tracking what changed.

The drift produces systems where substrate carries post-change state but change records reside elsewhere. The downstream consequences are predictable in shape if not always in timing: path traversal must cross system boundaries to follow antecedent references, breaking the substrate-only-paths property; change-related accountability questions cannot be answered from substrate alone, requiring federation across systems; regression tests cannot reference change events as first-class substrate content; and source-of-truth authority fragments across substrate (state) and external systems (history), undermining the source-of-truth commitment that the broader determinism contract rests on.

Naming Guarantee C as a standalone architectural commitment — with the four operational components made explicit, the limitations stated, the four adjacent-pattern distinctions drawn, the load-bearing connections enumerated, and the failure modes named — gives downstream implementations a precise specification of what change addressability the architecture requires. Subsequent decomposition notes formalize Guarantees D and E, the allowed non-determinism, and regression testing as standalone, completing the operational decomposition of the determinism contract.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Guarantee C — Substrate Changes Are Addressable: Standalone Treatment of Change Addressability in the Coordination Knowledge Substrate Pattern*. May 5, 2026. ORCID: 0009-0004-8065-3235.